

Arquitectura del Proyecto y Ejemplo de Implementación

Este proyecto es una **Single Page Application (SPA)** construida con **Vue 3**, **Vuetify** y **TypeScript**. No contiene código de backend (servidor API), por lo que la creación de un "Endpoint" real debe hacerse en el repositorio del servidor.

Arquitectura Frontend

La estructura del proyecto es **Modular**:

- **src/modules/**: Aquí reside la lógica de negocio dividida por funcionalidad (ej. catalog, auth, inventory).
 - Cada módulo contiene sus propios services (llamadas a API), views (componentes principales), types y routes.
- **src/core/**: Contiene utilidades compartidas.
 - **src/core/api/client.ts**: Configuración central de **Axios** con interceptores para enviar el Token y el ID de la empresa en cada petición.

Ejemplo: Eliminar Producto con Confirmación de Administrador

Aunque no podemos crear el endpoint de backend aquí, este es el código frontend necesario para consumir ese endpoint imaginario.

1. Servicio (Frontend)

Crea o edita `src/modules/catalog/services/product.service.ts` para incluir la llamada a la API.

```
// src/modules/catalog/services/product.service.ts
import api from '@core/api/client' // Usando el alias configurado
import type { Product } from '@modules/catalog/types'

/**
 * Envía una petición para "eliminar" (desactivar) un producto.
 * Se requieren credenciales de administrador para confirmar la acción.
 */
export async function deleteProductWithAdminAuth(
  productId: number,
  adminUser: string,
  adminPass: string
): Promise<void> {
  // Asumimos que el backend espera este payload
  const payload = {
    status: 'INACTIVE', // Cambio de estado lógico
    admin_confirmation: {
      username: adminUser,
      password: adminPass
    }
  }

  // POST o PATCH dependiendo de tu backend.
  // POST es común para acciones complejas con confirmación.
  await api.post(`/products/${productId}/deactivate`, payload)
}
```

2. Componente (Vista)

En tu componente Vue (por ejemplo, en src/modules/catalog/views/ProductList.vue), usa **SweetAlert2** para pedir las credenciales y llamar al servicio.

```
<script setup lang="ts">
import Swal from 'sweetalert2'
import { deleteProductWithAdminAuth } from '@modules/catalog/services/product.service'

const handleDeleteProduct = async (product: any) => {
  // 1. Confirmación inicial y captura de credenciales del Admin
  const { value: formValues } = await Swal.fire({
    title: 'Confirmación de Administrador',
    html:
      '<p>Para eliminar este producto, ingrese credenciales de administrador.</p>' +
      '<input id="swal-input1" class="swal2-input" placeholder="Usuario Admin">' +
      '<input id="swal-input2" class="swal2-input" type="password" placeholder="Contraseña">',
    focusConfirm: false,
    showCancelButton: true,
    confirmButtonText: 'Confirmar y Eliminar',
    cancelButtonText: 'Cancelar',
    preConfirm: () => {
      return [
        (document.getElementById('swal-input1') as HTMLInputElement).value,
        (document.getElementById('swal-input2') as HTMLInputElement).value
      ]
    }
  })
}
```

```

    ]
  }
})

// Si el usuario cancela, formValues será undefined
if (!formValues) return

const [adminUser, adminPass] = formValues

if (!adminUser || !adminPass) {
  Swal.fire('Error', 'Debe ingresar usuario y contraseña', 'error')
  return
}

try {
  // 2. Llamada al servicio
  await deleteProductWithAdminAuth(product.id, adminUser, adminPass)

  // 3. Éxito
  Swal.fire('Eliminado', 'El producto ha pasado a estado inactivo.', 'success')

  // Aquí deberías recargar la lista de productos
  // fetchProducts()
} catch (error: any) {
  console.error(error)
  // Manejo de errores (ej. credenciales inválidas)
  const errorMsg = error.response?.data?.message || 'No se pudo eliminar el producto'
  Swal.fire('Error', errorMsg, 'error')
}
}
</script>

<template>
  <v-btn color="error" @click="handleDeleteProduct(item)">
    Eliminar
  </v-btn>
</template>

```

Nota para el Backend

Para que esto funcione, tu equipo de Backend debe crear un endpoint (ej. POST `/api/products/:id/deactivate`) que:

1. Reciba `admin_confirmation` en el cuerpo.
2. Verifique que esas credenciales pertenezcan a un usuario con rol 'ADMIN'.
3. Actualice el estado del producto a 'INACTIVE' en la base de datos.
4. Inserte un registro en la tabla de logs/auditoría con la fecha, el usuario que realizó la acción y el motivo.